

Predicting Fork-to-PR Conversion in Open-Source Repositories

Business Understanding

This phase focuses on understanding the project objectives from a business perspective and converting them into a data mining problem definition.

Determine Business Objectives

Background

Commercial context. Open-source software (OSS) organizations rarely sell the code itself. Revenue comes from the surrounding commercial layer: managed hosting, enterprise support, dual licensing, and premium features in an open-core model. In each case, the scale and engagement of the external developer community are central value drivers.

What a fork signals. A *fork* on GitHub can reflect one of two qualitatively different patterns:

1. **Contributing use.** The developer returns work to upstream through a pull request, which strengthens the project and the community.
2. **Private downstream use.** The fork is maintained on its own with no pull request to upstream: the project delivers value to the user but receives no contribution back.

Role of this project. The balance between these two patterns is not measured systematically today. *GH Archive* captures the public GitHub event stream and makes it possible to estimate fork-to-contribution ratios at repository scale. This project defines that measurement and trains a predictive model linking repository-level signals to each outcome.

Business Objectives

Primary objective. Determine, for a sampled set of GitHub repositories, what fraction of forks result in pull requests back to upstream, and identify which repository-level characteristics predict high or low contribution conversion.

Primary users of the analysis

- Open-source program managers who oversee community health and engagement strategy.
- Product strategy teams that need to understand how their open-source assets are being adopted and extended.
- Developer relations teams that prioritize outreach based on repository contribution potential.

Decision supported by this analysis. Understanding which types of repositories attract contributing forks versus passive downstream forks allows stakeholders to prioritize community investment, documentation effort, and contribution tooling for the repositories where upstream engagement is most likely.

Related business questions

- What fraction of forks for a given repository leads to at least one pull request within a defined period?
- What distinguishes repositories where forks tend to become contributions from those where they do not?
- Are there repository segments where conversion is consistently low despite high fork activity?

Business Success Criteria

The project outcome is considered successful if it produces:

- A reliable and reproducible repository-level fork-to-PR conversion metric computed from public event data.
- A predictive model that identifies high- and low-conversion repositories with better accuracy than a naive baseline.
- An interpretable set of repository features associated with conversion outcomes, reviewable by non-technical stakeholders.
- A documented pipeline that can be re-run on new time periods.

Subjective judgment of output usefulness rests with the Business Analyst and the Business Unit.

Assess Situation

Inventory of Resources

Personnel

- Business Unit.
- Project Manager.
- Data Scientist.
- ML Engineer.
- Business Analyst.

Data

- GH Archive public dataset in BigQuery (`githubarchive.month.*`).
- Core events: `ForkEvent`, `PullRequestEvent`.
- Context events for feature engineering: `PushEvent`, `WatchEvent`, `IssuesEvent`, `IssueCommentEvent`, `ReleaseEvent`.

Computing and software

- BigQuery for SQL extraction and aggregation.
- Python stack (pandas/polars, scikit-learn, Jupyter) for modeling and evaluation.

Requirements, Assumptions, and Constraints

Requirements

- Results must be interpretable via thresholds, feature rankings, or trend summaries.
- Results must be reproducible: identical inputs produce identical outputs; all steps versioned.
- Data is publicly available under open access; no legal restrictions on analytical use.
- Scope must be bounded: observation window, sampling frame, and label definition fixed before modeling.
- Outputs must include uncertainty ranges from validation folds.

Assumptions

- A fork is marked as converted when at least one PR is observed that can be reasonably matched to the fork within the chosen window. Due to the absence of explicit linkage, this relationship will be approximated using heuristic matching.
- Multi-period sampling is more representative than a single-day snapshot.
- Repository-level aggregation reduces event-level noise.

Constraints

- **Fork-to-PR linkage is indirect.** The relationship must be inferred by matching repository identities across event types; no explicit link is stored in GH Archive.
- **Upstream metadata is sparse.** Static repository fields appear only when a pull request event is recorded. Repositories that never appear in such events must be described using event-aggregate features only.
- **Raw data volume is large.** One day of GH Archive is about 3 GB compressed. A three-month fork anchor plus a 90-day observation window spans roughly 270 GB compressed. All aggregation must run inside BigQuery before any export.
- **Class imbalance is expected.** Observed same-day fork-to-PR rate is 3–4%; even at 90 days the positive class will be a small minority.

Risks and Contingencies

- **Sampling bias across repository sizes.**
 - **Contingency:** Use stratified sampling by activity/popularity bands and report sample composition.
- **Noisy fork-to-PR matching.**
 - **Contingency:** Validate joins on sampled records and run consistency checks.
- **Temporal drift and seasonality.**
 - **Contingency:** Use time-based validation and compare multiple periods.
- **Over-interpretation of correlational results.**
 - **Contingency:** Report uncertainty and explicitly state non-causal interpretation.

Terminology

Business terminology

- **Fork:** a copy of a repository created by an external developer under their own account, typically to experiment or contribute.
- **Upstream repository:** the original repository from which a fork is derived.
- **Contribution:** an externally developed change submitted back to the upstream repository, typically as a pull request.

- **Contribution conversion:** the outcome where a fork leads to at least one pull request to upstream.
- **Fork-to-PR conversion rate:** the fraction of forks for a repository that resulted in at least one upstream pull request within a defined observation period.
- **Community health:** the degree to which a repository attracts active, contributing external developers.

Data mining terminology

- **ForkEvent:** a GH Archive event record that captures the creation of a fork. Used to identify fork instances and their upstream repositories.
- **PullRequestEvent:** a GH Archive event record that captures pull request actions. Used to detect whether a fork produced an upstream contribution.
- **Observation window:** the post-fork time interval within which a PR must appear for the fork to be labeled as converted. Example: 90 days after fork creation.
- **Repository-level aggregation:** grouping individual fork/PR event pairs by upstream repository to compute a conversion rate as a modeling target.
- **Stratified sampling:** selecting repositories proportionally across activity or popularity bands to ensure the sample is representative of the full distribution.

Costs and Benefits

Project costs

- **Data extraction:** design and execution of BigQuery SQL pipelines over multi-month GH Archive tables; estimated 2–4 major extraction runs with iterative refinement.
- **Label construction and validation:** fork-to-PR join logic, observation window tuning, and manual spot-checks on matched pairs to verify entity linkage correctness.
- **Feature engineering:** aggregation of event-based signals at the repository level (e.g., pushes, watches, issues) and construction of derived features (e.g., PR-to-fork ratios).
- **Modeling and evaluation:** training and cross-validating regression and classification baselines; hyperparameter search and ranking metric computation.
- **Reporting:** interpretation of model outputs, documentation of limitations, and production of final deliverables.
- **Infrastructure:** BigQuery query processing cost scales with the volume of scanned data; estimated low-to-moderate usage given partitioned queries and table wildcards.

Potential business benefits

- **Quantified community health signal:** for the first time, stakeholders obtain a concrete numeric measure—fork-to-PR conversion rate per repository—rather than relying on subjective community perception.
- **Prioritized portfolio view:** repositories can be ranked by expected contribution conversion, directing limited community investment toward projects with the highest return on engagement.

- **Early detection of passive-use segments:** repositories with many forks but low conversion are identified systematically, flagging potential gaps between what the project offers and what downstream users actually need.
- **Reusable pipeline:** the BigQuery extraction and feature construction logic can be re-run on any future period with no redesign, reducing the marginal cost of each subsequent analysis cycle.

Cost benefit comparison Under the assumptions below, expected quarterly benefit exceeds recurring quarterly operating cost and can recover setup cost within the first one to two quarters.

Rough quantitative estimate (illustrative assumptions)

- Repositories under active monitoring each quarter: 300.
- Average enterprise upgrade gross margin per converted repository: \$40.
- If conversion risk is not detected early, expected margin impact on at-risk repositories: 10–15% (= \$4–\$6 per repository).
- The analysis gives one-quarter earlier warning for 25–35% of at-risk repositories.

Estimated quarterly benefit range

- $300 \times (0.25 \text{ to } 0.35) \times (\$4 \text{ to } \$6) \approx \$300 \text{ to } \$630$ per quarter.

Estimated project cost range

- Initial setup: 20 analyst hours.
- Quarterly refresh: 4 analyst hours.
- Assuming \$20/hour internal cost: initial \$400, then \$80 per quarter.

Annualized benefit (\$1,200–\$2,520) is above annualized operating cost after setup (\$320/year), and setup cost can be recovered within the first one to two quarters under these assumptions.

Determine Data Mining Goals

Data Mining Goals

- Build repository-level target variables from fork-to-PR conversion in fixed windows.
- Train either conversion-rate regression or bucketed conversion classification models.
- Produce ranking outputs for high vs. low expected conversion repositories.
- Quantify feature influence using interpretable model diagnostics.

Data Mining Success Criteria

- **Conversion rate prediction:** mean absolute error on held-out repositories must be lower than the MAE of the mean-prediction baseline; R^2 must be positive on the test fold.
- **Classification:** macro-averaged F1 score must exceed the stratified random baseline by at least 5 percentage points.
- **Ranking quality:** precision@10% must exceed the rate expected under random ranking; NDCG or lift@k reported for the top decile.
- **Validation stability:** performance metrics must not degrade by more than 15% between the best and worst time-based validation fold, confirming that results are not driven by a single time period.
- **Feature interpretability:** the top 5 features by importance must have a plausible directional relationship with conversion that can be explained to a non-technical audience without further analysis.
- **Sensitivity check:** conversion rates computed under two alternative observation windows (e.g., 30 days and 90 days) must yield qualitatively consistent rankings; large rank reversals flag a fragile label definition.

Produce Project Plan

Project Plan

Stage 1: Data understanding

Duration: 1 week.

Resources required: Data Scientist, Business Analyst, BigQuery.

Inputs: GH Archive day/month tables, event schemas.

Outputs: schema notes, feasibility checks, sampled repository list.

Dependencies: BigQuery access and agreement on the event types and time range to be queried.

Decisions made at this stage: anchor period (3 calendar months of `ForkEvent`); observation window candidates (30, 60, 90 days post-fork); stratified sampling bands (fork-count quintiles); minimum fork threshold per repository.

Stage 2: Data preparation

Duration: 2 weeks.

Resources required: Data Scientist, ML Engineer.

Inputs: sampled repositories, fork and PR event extracts.

Outputs: labeled repository-level dataset and feature table.

Dependencies: approved matching logic and observation windows.

Decisions made at this stage: final observation window (selected from candidates); feature groups — event-aggregate features (push rate, watch rate, issue rate, PR rate) and static metadata from `pull_request.base.repo` where available; train/test split boundary (chronological: first 2 months train, third month test).

Stage 3: Modeling

Duration: 2 weeks.

Resources required: Data Scientist, ML Engineer.

Inputs: training and validation datasets.

Outputs: baseline and tuned models, ranking outputs.

Dependencies: finalized feature set and validation protocol.

Decisions made at this stage: model candidates — mean-rate baseline, Logistic Regression, Random Forest, Gradient Boosting; target formulation — conversion rate regression or low/medium/high bucket classification; class imbalance handling via class weights and threshold tuning.

Stage 4: Evaluation

Duration: 1 week.

Resources required: Data Scientist, Business Analyst, Project Manager.

Inputs: trained models, hold-out predictions.

Outputs: metric report, error analysis, interpretation summary.

Dependencies: completed modeling runs and evaluation scripts.

Evaluation strategy: chronological hold-out on the third month; primary metrics MAE and R^2 for regression, macro-F1 for classification, precision@10% for ranking; sensitivity analysis over the two observation window lengths; SHAP values for top-5 feature interpretation.

Stage 5: Deployment

Duration: 1 week.

Resources required: ML Engineer, Project Manager.

Inputs: final SQL scripts, model artifacts, documentation.

Outputs: reproducible pipeline and project handover package.

Dependencies: approval of evaluation results.

Decisions made at this stage: pipeline delivered as versioned BigQuery SQL scripts plus a Python notebook; model serialised with `joblib`; final report documents all parameter choices and their rationale.

Review points and large-scale iterations

- Review point after Stage 1: verify sampling representativeness and schema assumptions.
- Review point after Stage 2: verify label quality and feature completeness.
- Iteration loop between Stage 2 and Stage 3 if label window or joins need adjustment.
- Review point after Stage 4: decide whether model quality is sufficient or another iteration is required.

Schedule-risk dependencies (actions)

- **Risk:** delayed BigQuery extraction due to oversized scans. **Action:** partitioned queries and narrower pilot windows first.
- **Risk:** weak target signal in selected period. **Action:** extend anchor period and rebalance sampled repositories.
- **Risk:** unstable metrics across folds. **Action:** enforce rolling time splits and simplify feature set.

Initial Assessment of Tools and Techniques

- **Data extraction:** BigQuery SQL with table wildcards and JSON extraction functions.
- **Transformation:** SQL aggregations plus pandas/polars for final feature shaping.
- **Modeling:** scikit-learn linear and tree-based baselines.

- **Evaluation:** time-based validation, ranking metrics, and calibration diagnostics.

Data Understanding

Collect Initial Data

The initial data for this project was obtained from the public GitHub Archive dataset available in Google BigQuery. This dataset contains a continuous stream of GitHub activity events (e.g., forks, pull requests, pushes, and issues) and is updated hourly.

Data Sources and Access

- **Source:** GitHub Archive (public dataset in BigQuery)
- **Access method:** SQL queries executed via Google BigQuery
- **Storage:**
 - Final datasets were directly constructed in BigQuery
 - Exported to Google Colab in `.parquet` format:
 - * `activity_sample.parquet`
 - * `fork_pr_sample.parquet`

Data Acquisition Process

The data collection process was implemented as a single-stage sampling pipeline directly on top of the raw GitHub Archive tables.

1. Extraction of event data from BigQuery for the year 2025
2. Filtering of relevant event types:
 - ForkEvent
 - PullRequestEvent
 - WatchEvent
 - PushEvent
 - IssuesEvent
 - IssueCommentEvent
 - ReleaseEvent
3. Construction of a sampled activity dataset (`activity_sample`), where:
 - repository-level activity metrics are aggregated
 - repositories are filtered by minimum fork activity
 - only active repositories (at least one interaction signal) are retained
 - a random sample of repositories is selected
4. Construction of a fork and pull request dataset (`fork_pr_sample`) using the same set of repositories to ensure consistency between features and target variables

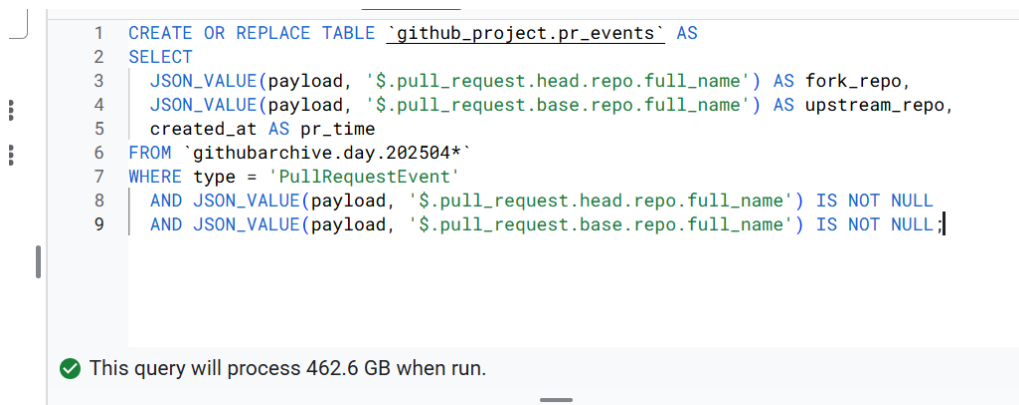
Key Design Decisions

Several important design decisions were made to balance analytical goals and computational constraints:

- **Anchor periods:** Fork events are restricted to two 3-month windows (January–March and July–September), which define the reference periods for analysis
- **Observation window:** Pull request events are collected over extended seasonal windows (January–June and July–December) to allow for delayed contributions after forks
- **Minimum fork threshold:** Only repositories with more than 5 forks within the anchor period are retained to reduce noise from low-activity repositories
- **Activity filtering:** Repositories must exhibit at least one interaction signal (watches, pushes, issues, issue comments, or releases)
- **Sampling strategy:** A random sample of repositories is drawn after filtering to reduce data volume while preserving diversity
- **Seasonality handling:** Two distinct seasonal periods (winter–spring and summer–autumn) are used to reduce temporal bias and capture variation in developer activity

Problems Encountered and Solutions

- **Large data volume:** The GitHub Archive dataset is extremely large (tens of gigabytes for a single year). This was addressed by applying early filtering and sampling directly in BigQuery queries, avoiding materialization of full raw datasets.
- **Storage limitations:** The BigQuery sandbox imposes strict storage limits. This was mitigated by constructing sampled datasets directly, instead of storing intermediate full-scale tables.
- **High cost of payload field:** The `payload` field significantly increases data size [Figure 1]. It was excluded, and only structured fields (e.g., `repo.name`, `actor.login`) were used.
- **Sampling bias:** GitHub data is highly skewed toward popular repositories. This was mitigated by applying a minimum activity threshold and random sampling after filtering.
- **Temporal drift and seasonality:** Developer activity varies over time. This was addressed by explicitly defining two seasonal periods and aligning all features and targets with these periods.



```
1 CREATE OR REPLACE TABLE `github-project.pr_events` AS
2 SELECT
3   JSON_VALUE(payload, '$.pull_request.head.repo.full_name') AS fork_repo,
4   JSON_VALUE(payload, '$.pull_request.base.repo.full_name') AS upstream_repo,
5   created_at AS pr_time
6 FROM `githubarchive.day.202504*`
7 WHERE type = 'PullRequestEvent'
8    AND JSON_VALUE(payload, '$.pull_request.head.repo.full_name') IS NOT NULL
9    AND JSON_VALUE(payload, '$.pull_request.base.repo.full_name') IS NOT NULL;
```

✓ This query will process 462.6 GB when run.

Figure 1: Influence of including the payload field on query size

Outcome

As a result of the data collection process, two main datasets were prepared:

- **activity_sample:** repository-level activity metrics, including watches, pushes, issues, issue comments, and releases
- **fork_pr_sample:** structured arrays of fork and pull request events with user identifiers and timestamps, aligned with the sampled repositories and seasonal periods

These datasets form the basis for subsequent data understanding, exploration, and preparation stages.

Describe Data

In this stage, the acquired datasets are examined at a surface level in order to understand their structure, size, and general properties. The goal is to assess whether the collected data is suitable for further analysis and modeling.

Data Format and Structure

Both datasets obtained during the data collection stage are stored in **.parquet** format and loaded into a tabular structure using pandas DataFrames. Each dataset represents repository-level observations, with rows corresponding to repository-season pairs.

The **activity_sample** dataset contains aggregated numerical metrics describing repository activity, while the **fork_pr_sample** dataset contains nested structures representing lists of events (forks and pull requests) for each repository.

The data types of the columns reflect this difference. In the activity dataset, numerical columns such as **watches**, **pushes**, **issues**, **issue_comments**, and **releases** are stored as integer types, while identifiers such as **repo_name** and **season** are stored as strings. In contrast, the fork/PR dataset stores event data in object-type columns, as they contain structured arrays of user and timestamp information.

Data Size and Dimensions

Both datasets contain 100,000 rows, corresponding to the sampled repository-season combinations.

The activity dataset [Figure 2] contains 9 columns:

- **repo_name**
- **season**
- **repo_url**
- **public**
- **watches**
- **pushes**
- **issues**
- **issue_comments**
- **releases**

	repo_name	season	repo_url	public	watches	pushes	issues	issue_comments	releases
0	GoodRequest/BackendAssignment-Fitness	winter_spring	https://api.github.com/repos/GoodRequest/Backe...	True	0	1	0	0	0
1	NREL/DE	winter_spring	https://api.github.com/repos/NREL/DE	True	0	1	0	0	0
2	danhpaiva/class-psc-algorithm-00	summer_autumn	https://api.github.com/repos/danhpaiva/class-p...	True	0	1	0	0	0
3	NaturalCoder/TI96	winter_spring	https://api.github.com/repos/NaturalCoder/TI96	True	0	25	0	0	0
4	josedom24/ic-html5	winter_spring	https://api.github.com/repos/josedom24/ic-html5	True	0	2	0	0	0

Figure 2: Head of activity table

The fork/PR dataset [Figure 3] contains 4 columns:

- repo_name
- season
- forks
- prs

	repo_name	season	forks	prs
0	Quitten/Autorize	summer_autumn	[{'event_time': 2025-09-13 12:19:55+00:00, 'ty...	[{'event_time': 2025-09-01 19:31:59+00:00, 'ty...
1	imzyb/MiSub	summer_autumn	[{'event_time': 2025-09-05 07:39:27+00:00, 'ty...	[{'event_time': 2025-11-01 07:20:49+00:00, 'ty...
2	facebookresearch/pytorch3d	summer_autumn	[{'event_time': 2025-09-22 18:15:37+00:00, 'ty...	[{'event_time': 2025-08-01 14:46:39+00:00, 'ty...
3	meltlabs/melty	summer_autumn	[{'event_time': 2025-08-17 20:25:43+00:00, 'ty...	[]
4	GouvernementFR/dsfr	summer_autumn	[{'event_time': 2025-09-01 14:44:37+00:00, 'ty...	[{'event_time': 2025-11-20 15:32:26+00:00, 'ty...

Figure 3: Head of fork_pr table

The memory footprint of the datasets is relatively small:

- Activity dataset: approximately 29 MB
- Fork/PR dataset: approximately 40 MB

This confirms that the data has been successfully reduced from the original large-scale source to a manageable size suitable for local analysis.

Field Description

The `activity_sample` dataset includes the following types of variables:

- **Identifiers:** repo_name, season
- **Metadata:** repo_url, public
- **Activity metrics:** watches, pushes, issues, issue_comments, releases

The `fork_pr_sample` dataset includes:

- **Identifiers:** repo_name, season
- **Event structures:**
 - forks: list of fork events (user identifier and timestamp)
 - prs: list of pull request events (user identifier and timestamp)

Thus, the datasets complement each other: one provides aggregated activity signals, while the other preserves detailed event-level information required for modeling contribution dynamics.

Initial Assessment

Overall, the acquired data satisfies the requirements for further analysis. The datasets are well-structured, compact, and contain both aggregated activity signals and detailed event-level information necessary for analyzing the relationship between repository activity and contribution behavior.

Explore Data

At this stage, exploratory data analysis was conducted to understand the distribution of key variables, relationships between features, and behavioral patterns relevant to the project objectives. The focus was placed on identifying signals that may influence the likelihood of fork-to-pull-request conversion.

Distribution of Activity Metrics

The distributions of repository activity metrics (`watches`, `pushes`, `issues`, `issue_comments`, `releases`) reveal a highly skewed structure over the 100,000 repository–season rows in the activity table. Summary statistics (e.g. `describe()`) show means far above medians, as expected under heavy tails—for `watches` alone, the mean is 120.7 while the 50th percentile is 27.

While median values remain relatively low, the maximum values are extremely large, indicating the presence of a small number of highly active repositories dominating the distribution. Per-metric medians are 27, 4, 1, 2, and 0 (for the five fields in the same order), whereas maxima are 79,049, 340,799, 48,592, 199,042, and 2,143.

Outlier analysis confirms the existence of extreme repositories with disproportionately high activity across all metrics: 1,476 records (1.5% of the sample) pass the EDA high-activity rule (combined thresholds on `watches`, `pushes`, `issues`, `issue_comments`, and `releases`). This heavy right-skew is consistent across all activity features.

Implication: Logarithmic transformation or other scaling techniques will be required to stabilize variance and make patterns more detectable for modeling.

Seasonality Analysis

The dataset includes two seasonal segments: `winter_spring` (January–March) and `summer_autumn` (July–September).

The distribution across seasons is moderately imbalanced, but median activity values remain relatively consistent between the two periods across all metrics.

Implication: Seasonality does not significantly affect typical repository behavior, allowing both periods to be used jointly in modeling.

Public vs Private Repositories

All repositories in the dataset are marked as public.

Implication: The `public` attribute does not provide useful variation and can be excluded from the modeling stage.

Fork and Pull Request Distributions

The fork and pull request counts also exhibit strong skewness.

A substantial proportion of repositories receive forks but no pull requests:

- Approximately 34% of repositories have zero pull requests despite having forks

Implication: Conversion from fork to pull request is not guaranteed and represents a meaningful and non-trivial prediction target.

Repeated User Activity

Analysis of user behavior shows that most users fork a repository only once. However, pull request activity is significantly more intensive.

Repositories frequently contain users submitting multiple pull requests, indicating that contribution behavior is concentrated among a smaller number of highly active contributors.

Implication: Contribution dynamics are driven more by engaged contributors than by the number of casual users.

Relationship Between Activity and Contributions

Correlation analysis between activity features shows generally weak linear relationships[Figure 4]:

- The strongest correlations are observed between interaction-based metrics (e.g., issues and issue comments)
- Popularity metrics such as watches show very weak correlation with other features

<code>pr_per_fork</code>	<code>1.000000</code>
<code>issue_comments</code>	<code>0.213205</code>
<code>pushes</code>	<code>0.137040</code>
<code>issues</code>	<code>0.082955</code>
<code>releases</code>	<code>0.071771</code>
<code>watches</code>	<code>-0.010566</code>

Figure 4: Correlation with target

Further analysis of the relationship between activity and contribution outcomes shows:

- `issue_comments` has the strongest positive association with conversion behavior
- `pushes`, `releases` and `issues` show weaker but still positive relationships
- `watches` (repository popularity) shows near-zero correlation with contribution outcomes

Implication: Interaction-based signals are more informative for predicting contributions than popularity-based metrics.

Log-Scale Analysis

Due to the strong skewness of the data, relationships were additionally analyzed in log scale.

Log-transformed scatter plots [Figure 5] reveal clear non-linear relationships between activity metrics and fork behavior.

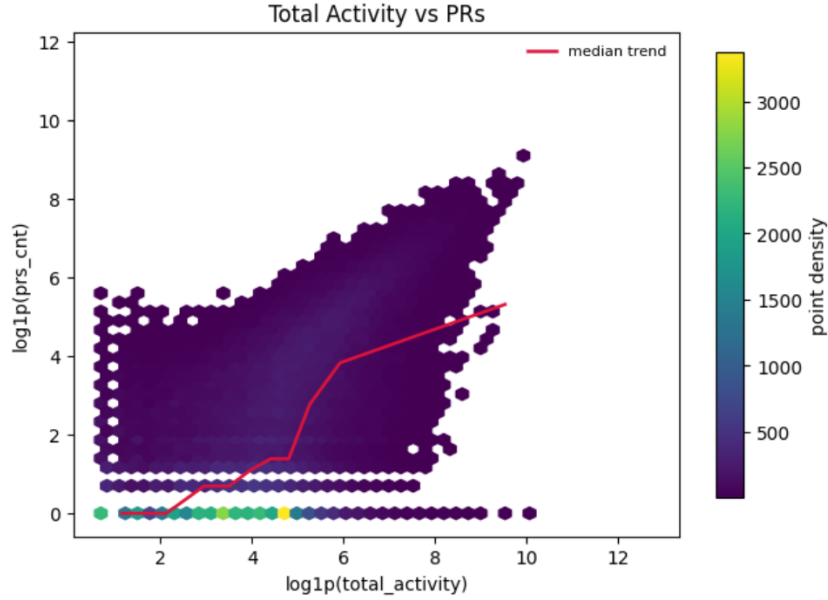


Figure 5: Activity vs PRs (log scale)

Implication: Modeling approaches should account for non-linearity, either through transformations or non-linear models.

Derived Metric: PR per Fork

A derived metric was introduced to estimate repository-level conversion intensity:

$$\text{PR per Fork} = \frac{\text{PR count}}{\text{Fork count}}$$

The distribution of this metric is highly skewed:

- Median is low—the 50th percentile of `pr_per_fork` is about 0.23 across $n = 100,000$ repository-season rows, indicating that most pairs have limited conversion intensity in this ratio
- The mean is about 3.98, much larger than the median: the 75th percentile is only 2.0 while the standard deviation is about 18.7, so a small share of outlier points pulls the mean upward
- Maximum `pr_per_fork` reaches about 2,595—extremely large values driven by very small fork counts in the denominator relative to many pull requests, which distorts the ratio

Implication: While the metric captures conversion behavior, it is unstable for small fork counts and sensitive to extreme values. Careful handling (e.g., clipping or transformation) is required.

Key Findings

The exploratory analysis revealed several important characteristics of the data:

- All activity and contribution variables exhibit strong right-skew and extreme outliers
- A significant share of repositories does not convert forks into pull requests
- Contribution behavior is weakly correlated with activity metrics, indicating a non-trivial prediction task
- Interaction-based signals (especially issue discussions) are more predictive than popularity metrics
- Non-linear relationships are present, suggesting the need for transformation or non-linear modeling approaches

Verify Data Quality

In this stage, the quality of the collected datasets was assessed to identify potential issues that could affect downstream analysis and modeling. The evaluation focused on completeness, consistency, correctness, and the presence of anomalies.

Missing Values

Both datasets were examined for missing values across all columns.

No missing values were found in the `activity_sample` dataset. Similarly, the `fork_pr_sample` dataset contains no missing values at the top level.

To further validate completeness, nested event structures were inspected by expanding the `forks` and `prs` arrays.

The analysis shows:

- No missing user identifiers within fork events
- Missing entries in pull request arrays correspond to repositories with no pull requests rather than incomplete data

Implication: The dataset is complete; apparent missing values in nested structures reflect absence of events rather than data quality issues.

Data Consistency

Consistency checks were performed across datasets:

- No duplicate `repo_name-season` pairs were found in either dataset
- The set of repositories is consistent between `activity_sample` and `fork_pr_sample`

Implication: The datasets are properly aligned and can be safely joined without loss of information.

Validity of Values

All numerical activity metrics (`watches`, `pushes`, `issues`, `issue_comments`, `releases`) were verified to ensure they contain valid values.

- No negative values were detected across any metric

Time-based validation of fork events shows the expected temporal coverage:

- From: January 2025
- To: September 2025

This corresponds to the defined anchor periods used during data extraction.

Implication: The temporal filtering logic was applied correctly, and all values fall within expected ranges.

Outliers and Extreme Values

Quantile analysis [Figure 6b] confirms the presence of extreme values across all activity metrics.

Upper quantiles (e.g., 99th and 99.9th percentiles) are significantly higher than median values [Figure 6a], indicating heavy-tailed distributions.

Implication: These extreme values are not data errors but reflect the natural structure of GitHub activity, where a small number of repositories dominate engagement. However, they may disproportionately influence model training and will require appropriate handling.

	watches	pushes	issues	issue_comments	releases
count	1476.0	1476.0	1476.0	1476.0	1476.0
mean	2181.607724	2175.53252	424.816396	1960.968835	70.550813
std	4850.75688	10828.241334	1883.855765	7604.560614	165.27907
min	0.0	0.0	0.0	0.0	0.0
25%	42.0	101.0	14.0	56.0	1.0
50%	223.0	295.5	69.5	257.5	43.0
75%	3169.0	1047.0	263.5	1054.75	72.0
max	79049.0	340799.0	48592.0	199042.0	2143.0

(a) Summary statistics of activity metrics

	watches	pushes	issues	issue_comments	releases
0.950	400.0	269.05	80.0	263.0	7.0
0.990	1536.02	1192.02	301.0	1169.01	33.0
0.999	7814.032	7404.009	1376.029	6779.099	170.001

(b) Outliers in activity metrics

Figure 6: Distribution and extreme values of activity features

Feature Relationships and Redundancy

Additional validation was performed to assess relationships between features.

Pearson correlations on the five activity counters ($n = 100,000$ repository-season rows) are uniformly modest: the largest off-diagonal coefficient is between **issues** and **issue_comments** ($r \approx 0.22$). The next-largest are **pushes** vs. **issue_comments** ($r \approx 0.16$) and **watches** vs. **issues** ($r \approx 0.11$). Every other pair is below 0.10 (for example, **watches** vs. **issue_comments** is about 0.09, and **pushes** vs. **issues** about 0.06). Interaction-related fields therefore co-move slightly more with one another than **watches** does with the rest, but even the strongest linear link is weak.

No pair approaches a correlation near one, so there is no sign of strong multicollinearity among these engineered activity features.

Implication: Features provide largely complementary information and are suitable for joint use in modeling without aggressive dimensionality reduction driven by redundancy alone.

Target Stability Check

The derived metric **pr_per_fork** was analyzed to assess stability.

The distribution is highly skewed, with:

- Low median values
- High variance
- Extreme maximum values caused by small fork counts combined with multiple pull requests

Implication: The target variable is sensitive to outliers and may require transformation or alternative formulations during modeling.

Overall Assessment

The data demonstrates high overall quality:

- No missing values at the table level

- No duplicate records at the repository-season level
- Consistent structure across datasets
- Valid numerical and temporal values

The main characteristics identified are:

- Presence of extreme values (outliers)
- Highly skewed distributions
- Non-linear relationships between variables

These characteristics are expected in real-world GitHub data and do not represent data quality issues. The dataset is suitable for further analysis and modeling, provided that appropriate preprocessing techniques are applied.

Data Preparation

The data preparation phase covers all activities to construct the final dataset for modeling from the initial raw data.

Select Data

Data Selection During Extraction

Selection during extraction already fixed the GitHub Archive schema: retained fields include repository and actor identifiers, event type and time, repository URL, visibility flag, and derived season; heavy or redundant fields (for example payload and most nested metadata) were excluded at extraction time. Event types used for features and targets include ForkEvent, PullRequestEvent, WatchEvent, PushEvent, IssuesEvent, IssueCommentEvent, and ReleaseEvent, with seasonal windows and repository filters as documented in the earlier report.

Data Selection During EDA

The public field exhibits no useful variation in this sample and is removed from the modeling copy of the activity table. The `repo_url` field is removed from the modeling copy because it is not used as a predictive feature in the current design (it can be rejoined externally if static metadata is added later). No repository-season rows are dropped at this step: the sampled keys from BigQuery are preserved; alignment with labels is enforced later with an inner join. In code, a copy `prep_activity` is built from activity after dropping public and `repo_url`, leaving identifiers `repo_name` and season plus the five activity counters (watches, pushes, issues, issue comments, releases).

Clean Data

Extreme values on activity metrics were documented as legitimate heavy tails rather than automatic data errors. The cleaning strategy is therefore tail reduction, not row deletion.

For each of the five activity counters, values are winsorized at the 99th percentile (`WINSOR_Q = 0.99`). Original columns are retained; capped values are stored in `*_winsor` columns. Before quantiles and clipping, series are cast to float64 so nullable integer dtypes from Parquet do not conflict with fractional quantile thresholds. Fork and PR event lists used for labels are not winsorized; labeling uses raw events and timestamps. This balances numerical stability for modeling with preservation of the full sample and of event-level detail for the label logic.

Construct Data

Sampling Strategy

The unit of analysis is one repository in one season. Each row in the integrated table corresponds to that pair. The sample size and inclusion rules follow the extraction query (e.g. the 100,000-row cap and filters applied in the warehouse). This phase does not subsample rows for class balance or stratification; the exported table is the full set of repository–season observations that pass the merge and labeling logic.

Matching Forks to Pull Requests

A fork counts as converted if there is at least one pull request whose `user_login` matches the fork author, whose timestamp is strictly after the fork, and which occurs within `OBSERVATION_DAYS` days after the fork (default 90 in the implementation). Events on or before the fork, or after the window, are ignored for conversion. Per repository and season the notebook computes `n_forks`, `n_converted_forks`, `fork_conversion_rate` (converted forks divided by fork count, not total PR events divided by forks without matching), and `n_pr_events` as a count of PR events in the cell.

Derived Features

For each activity counter, `log1p` is applied to both the raw values and the winsorized (`*_winsor`) versions, giving `log1p_*` columns suited to skewed nonnegative counts. Original counts and winsorized columns remain in the table alongside these transforms.

Generated Records

Each generated record is one flat row for a repository–season pair:

Only fork–PR pairs that satisfy the matching rules and fall inside the observation window increase `n_converted_forks` and `fork_conversion_rate`. The export stores scalars only; nested fork/PR lists are not written to the flat file. Rows are kept when there are no qualifying PRs: `n_converted_forks` may be zero; such rows are not removed from the dataset.

Integrate Data

The activity table (identifiers, activity counters, winsorized and log-transformed fields as built in the preparation pipeline) is joined to the per-repository–season label table on `repo_name` and season. The join is an inner merge with one-to-one key validation so each key appears at most once on both sides. The result is a single wide table with one row per repository and season, combining all feature columns with `n_forks`, `n_converted_forks`, `fork_conversion_rate`, `n_pr_events`, and any other label fields produced in the construction step.

Format Data

The merged table is ordered and typed for stable export and downstream use.

- `repo_id`: integer codes for `repo_name`; `repo_name` is kept for interpretation.
- `has_conversion`: binary flag, true when `n_converted_forks > 0`.
- Column order: identifiers and season, then `log1p_*` feature columns, then label columns (`n_forks`, `n_converted_forks`, `fork_conversion_rate`, `n_pr_events`, `has_conversion`), then any remaining columns (e.g. raw counts and `*_winsor`).
- Shuffle: rows are permuted with a fixed random seed for reproducibility.

- Output: primary file `modeling_dataset.parquet` when a Parquet engine is available; otherwise `modeling_dataset.csv`.

The file is row-complete for all keys that survive the inner join; missing-engine fallbacks only affect storage format, not the tabular content.